A

Special Project Mid-term Report

on

AI Voice-Controlled ERP Assistant

A Natural Language Interface for ERP Operations

by

Pranamya Mani Tripathi (24STUCHH010777)

Gone Srishanth Reddy (24STUCHH010428)

Harshulchandhra (24STUCHH010172)

Chintakindi Badrinath Goud (24STUCHH010308)

Under the guidance of

Sowjanya Ramisetty Mam

(Special Project Faculty)

Faculty of Science & Technology,

The ICFAI Foundation for Higher Education, Hyderabad

September, 2026

CONTENTS

Project Title Page

1. Introduction

1.1. Problem Definition

1.2. Objective(s) of the Project

1.3. Limitations of the Project

2. Literature Survey

2.1. Existing System

2.2. Limitations of Existing System

3. Proposed System - Methodology (Tools and Technologies)

3.1 System Architecture / Data flow diagrams

3.2 System Requirements & Specification

a. Software requirements

b. Hardware requirements

c. Data requirements (Inputs and Expected Outputs)

d. Description of Modules

e. UML Diagrams

4. Initial Implementation

5. References

1. Introduction

Enterprise Resource Planning (ERP) systems consolidate inventory, human resources, finance and sales data into one platform and are central to how organisations operate. However, their power comes with complexity: even a routine request, such as checking the stock of a single item or recording attendance, typically requires navigating several menus and screens and filling in structured forms. This project, the AI Voice-Controlled ERP Assistant, builds a conversational layer on top of an ERP so that users can perform such tasks by simply speaking or typing a sentence in plain language.

The system accepts a voice or text command, converts speech to text where necessary, extracts the user's intent and the relevant entities (item codes, employee IDs, time periods), engages in a short dialogue when information is missing or when data is about to be modified, executes the request against the ERP through a gateway, and returns the answer as text and optionally as synthesised speech. At this mid-term stage a working prototype has been built around a mock SQLite ERP with Inventory, Employees, Attendance and Sales modules.

1.1. Problem Definition

Enterprise ERP systems are powerful but notoriously difficult to use. Accessing even routine data requires navigating multiple screens, filling forms and understanding complex menu hierarchies. The specific problems addressed are:

- **Slow, error-prone navigation:** traditional ERP interaction is multi-step, and each step is an opportunity for a wrong click or a mistyped identifier.
- **Time spent on repetitive tasks:** routine operations such as stock look-ups, attendance marking and monthly sales totals consume a disproportionate amount of user time.
- **Steep learning curve:** non-technical users and field staff find complex interfaces difficult, and often need training before they can use the system.
- **Unreliable identifiers in speech:** alphanumeric codes such as "P-102" are hard for any speech recogniser to transcribe accurately, so a voice interface cannot depend on users reciting exact codes.
- **Safety of data changes:** a natural-language interface must not perform accidental write operations because of a misheard or ambiguous command.

1.2. Objective(s) of the Project

The primary objective is to enable users to interact with ERP functions using natural language, voice or text, instead of manually navigating the system. The specific objectives are:

- To design a layered architecture (User Interface, AI Processing, Integration and ERP Core) with a clear responsibility for each layer.
- To support both voice and text input interchangeably through the same processing pipeline.

- To implement intent recognition and entity extraction using a rule-based NLU engine, with an optional LLM-based engine (Ollama with Llama 3.2) for greater linguistic flexibility.
- To resolve item and employee names to ERP records using fuzzy matching, so that users need not remember exact record IDs.
- To implement a Dialog Manager that asks clarifying questions for missing information and demands explicit confirmation before any write operation.
- To provide a role-based access control (RBAC) gate in the ERP Gateway and optional sign-in, so that users can only perform operations permitted for their role.
- To build a mock SQLite ERP and a web front-end that demonstrate the complete pipeline end to end, in a design that can later be pointed at a real ERP such as Odoo.

1.3. Limitations of the Project

The current prototype has the following limitations, several of which are planned as future work:

- **Limited language variation in the default NLU:** the rule-based engine understands only phrasings that match pre-written regular-expression patterns. The LLM-based engine addresses this but requires a local Ollama installation and a downloaded model.
- **Mock ERP data:** the ERP Core is a SQLite database with a simplified schema and small seed data; it has not yet been integrated with a production ERP such as Odoo or SAP.
- **Prototype-grade security:** the optional authentication uses environment-configured users and in-memory sessions, which are lost on restart and not shared between server workers. Production use would require OAuth2/JWT with an established identity provider, rate limiting and hardening.
- **Session state in memory:** dialog state is held in a Python dictionary and would need a shared store such as Redis for multi-instance deployment.
- **Speech accuracy:** speech recognition quality depends on the microphone, background noise and the chosen engine; the voice path additionally requires ffmpeg, an STT engine and pyttsx3 to be installed.
- **Single language and small operation set:** only English and four core modules (inventory, attendance, sales, employees) with six intents are supported currently.

2. Literature Survey

2.1. Existing System

Conversational and voice interfaces for enterprise software have been explored from several directions. The main categories of existing systems relevant to this project are summarised below.

- **Consumer voice assistants (Amazon Alexa, Google Assistant, Apple Siri):** these combine speech recognition and natural-language understanding for general tasks such as reminders, media and smart-home control. They rely on cloud services and are not designed to query private enterprise databases.
- **Vendor-embedded ERP copilots (for example SAP Joule and Microsoft Copilot for Dynamics 365):** ERP vendors are adding generative-AI assistants to their own suites, allowing users to ask questions in natural language within that vendor's ecosystem.
- **Enterprise chatbot platforms (Oracle Digital Assistant, Rasa, Google Dialogflow, Amazon Lex):** these provide intent classification, entity extraction and dialog management toolkits which developers integrate with back-end systems through custom fulfilment code.
- **Speech-to-text engines (OpenAI Whisper, Vosk):** Whisper is a large-scale weakly supervised model that is robust across accents and noise [1], and Vosk is a lightweight offline toolkit suitable for local deployment [5].
- **Open-source language models for NLU (Llama 3.2 served through Ollama):** small language models can run locally and map free-form sentences to structured JSON intents and entities without any external API [2], [6].
- **Classical spoken dialogue system design:** the standard pipeline of speech recognition, language understanding, dialogue management and response generation is described in the speech-processing literature [3]; this project follows that structure.

2.2. Limitations of Existing System

Existing approaches have several shortcomings that motivate the proposed system:

- Traditional ERP user interfaces remain menu- and form-driven, requiring multiple steps and prior training for routine tasks.
- Consumer voice assistants cannot securely access company ERP data, and cloud-only services raise privacy concerns for internal business data.
- Vendor copilots are typically tied to a single vendor's platform and licensing, and are not easily adapted to smaller organisations or custom back-ends.
- General chatbot platforms provide building blocks but leave the ERP integration, safety controls and voice handling to be built by the developer.

- Most voice systems assume that codes and identifiers are transcribed correctly, and do not provide a fallback such as fuzzy name resolution with user confirmation when the recogniser makes mistakes.
- Many conversational systems do not enforce an explicit confirmation step before executing write operations, which risks unintended data changes.

The proposed system targets these gaps by offering a lightweight, pluggable and locally deployable assistant with a built-in Dialog Manager that guards write operations and tolerates imperfect speech recognition.

3. Proposed System - Methodology (Tools and Technologies)

The proposed system is a four-layer web application. A user speaks or types a command in the browser; the FastAPI backend passes it through speech-to-text (for audio), NLU, entity resolution and the Dialog Manager; the ERP Gateway then executes the validated request against the ERP database and formats a reply, which is returned to the browser as text and optionally as synthesised speech.

3.1 System Architecture / Data flow diagrams

System architecture

The architecture consists of four layers: (1) the User Interface Layer, a web page with text and voice input; (2) the AI Processing Layer, which contains speech-to-text, NLU, entity resolution and the Dialog Manager; (3) the Integration Layer, comprising the FastAPI backend and ERP Gateway with role-based access control; and (4) the ERP Core, a SQLite mock ERP with Inventory, Employees, Attendance and Sales modules.

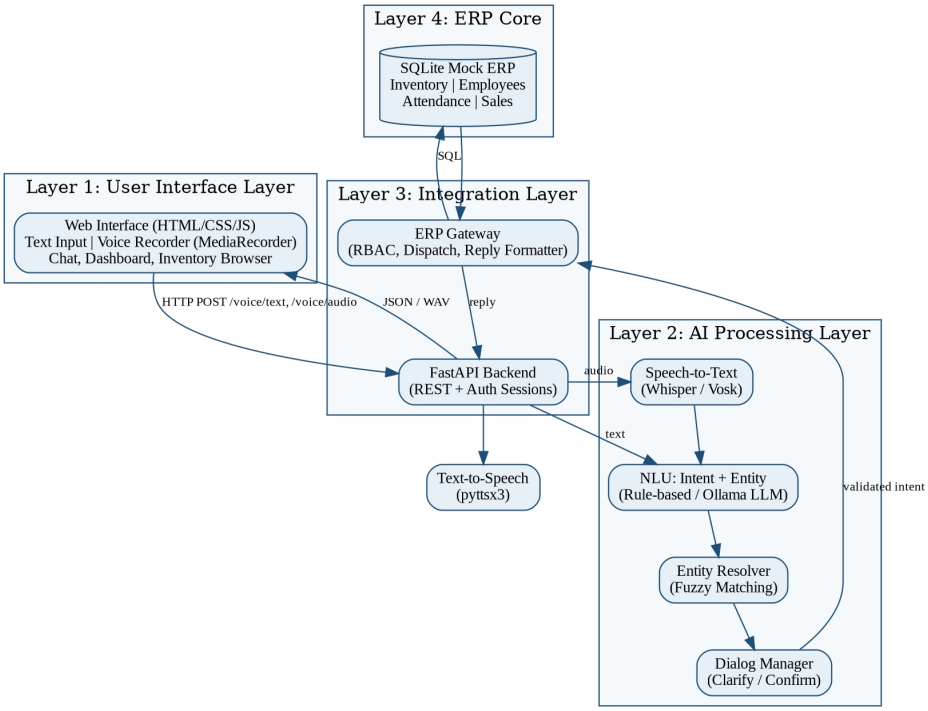


Figure 3.1: Layered system architecture of the AI Voice-Controlled ERP Assistant

End-to-end request flow

For the example command "Show stock for item P-102", the request passes through seven stages:

1. **Voice / Text Input:** the user speaks or types the command into the web interface.
2. **Speech-to-Text:** audio is converted to text by Whisper or Vosk after being normalised to 16 kHz mono; text input skips this stage and enters the NLU directly.
3. **NLU Processing:** intent = check_stock and entity item_id = P-102 are extracted (rule-based or LLM-based).
4. **Dialog Manager:** checks that all required entities are present and reliable, and requests clarification or confirmation if needed.
5. **ERP Gateway:** verifies the user's role and routes the validated request to the correct ERP function.
6. **Database Query:** SQLite returns the stock record for item P-102.
7. **Response:** the result is formatted as a sentence, displayed in the chat, and optionally converted to speech using pyttsx3.

Data flow diagram

Figure 3.2 shows the level-1 data flow diagram. Two data stores are involved: the ERP database (D1) and the in-memory session store (D2) that holds the dialog state such as pending intents, pending entities and awaiting confirmations.

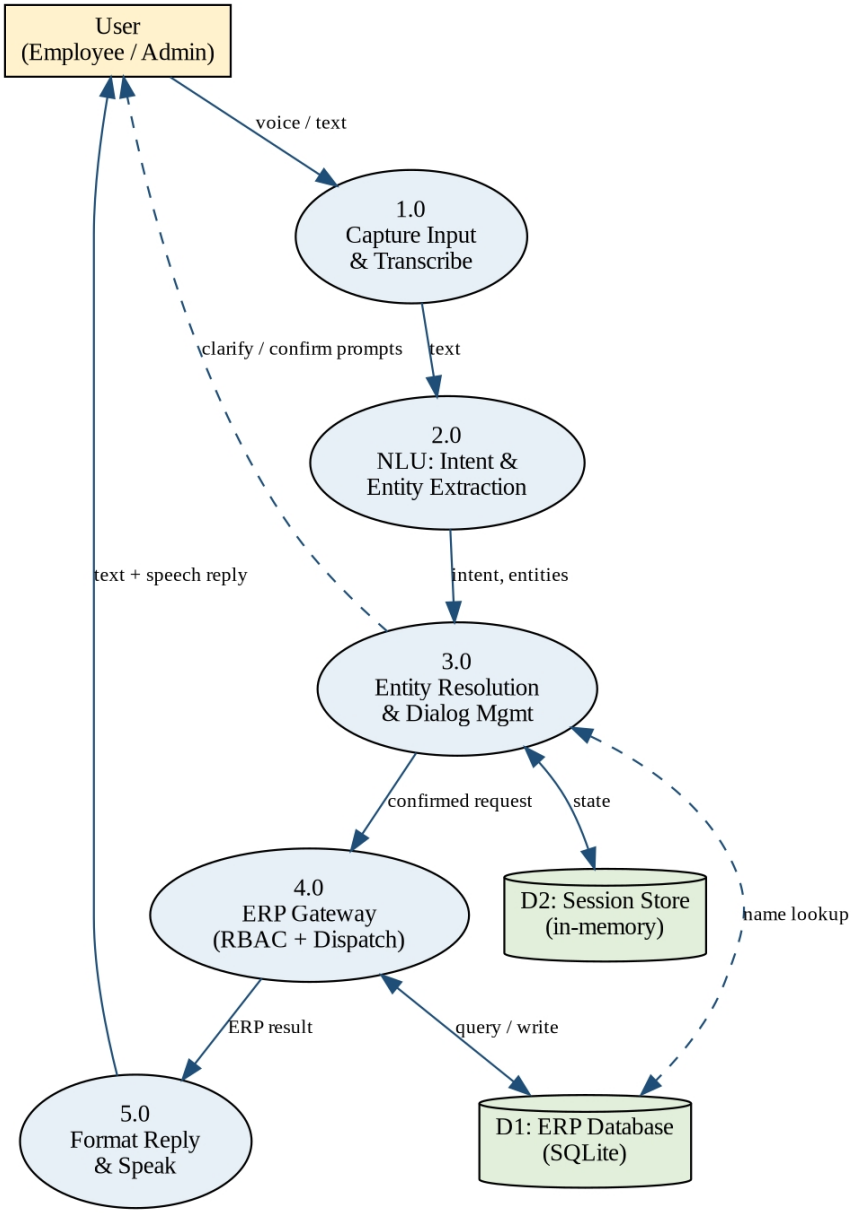


Figure 3.2: Level-1 data flow diagram

3.2 System Requirements & Specification

a. Software requirements

Table 3.1: Software requirements

Component	Requirement / Technology
Operating system	Windows 10/11, Linux or macOS
Programming language	Python 3.10 or later (type-hint syntax such as str | None is used)
Backend framework	FastAPI, Uvicorn (ASGI server), Pydantic, python-multipart
Database	SQLite (mock ERP schema), accessed through Python's sqlite3 module
Front-end	HTML, CSS and JavaScript (no build step); browser MediaRecorder API for microphone capture

Component	Requirement / Technology
NLU (default)	Rule-based regex/pattern matching (Python re module)
NLU (optional)	Ollama with the llama3.2:1b model; the requests library for local API calls
Entity resolution	difflib.SequenceMatcher (Python standard library) for fuzzy matching
Speech-to-Text (optional)	OpenAI Whisper (API or local) or Vosk (offline); ffmpeg for audio pre-processing
Text-to-Speech (optional)	pyttsx3 (offline)
Browser	Modern browser with microphone support (Chrome, Edge, Firefox)

b. Hardware requirements

Table 3.2: Hardware requirements

Component	Minimum / Recommended
Processor	Dual-core CPU (minimum); quad-core or better recommended when running the local LLM or Whisper
Memory (RAM)	4 GB minimum for the rule-based pipeline; 8 GB or more recommended for Ollama with Llama 3.2 1B
Storage	500 MB for the application and dependencies; about 1.3 GB additional for the Llama 3.2 1B model and further space for STT models
Audio devices	Microphone and speakers or headset (a headset microphone improves recognition accuracy)
Network	Not required for the offline configuration; internet needed only for the Whisper API option and one-time model downloads
GPU	Not required; models run on CPU

c. Data requirements (Inputs and Expected Outputs)

Inputs are spoken or typed natural-language commands. Voice input is uploaded as an audio file (.wav, .mp3 or browser recording) to the /voice/audio endpoint, and text input is sent as JSON with a session identifier to /voice/text. The ERP data are stored in four SQLite tables (Figure 3.3).

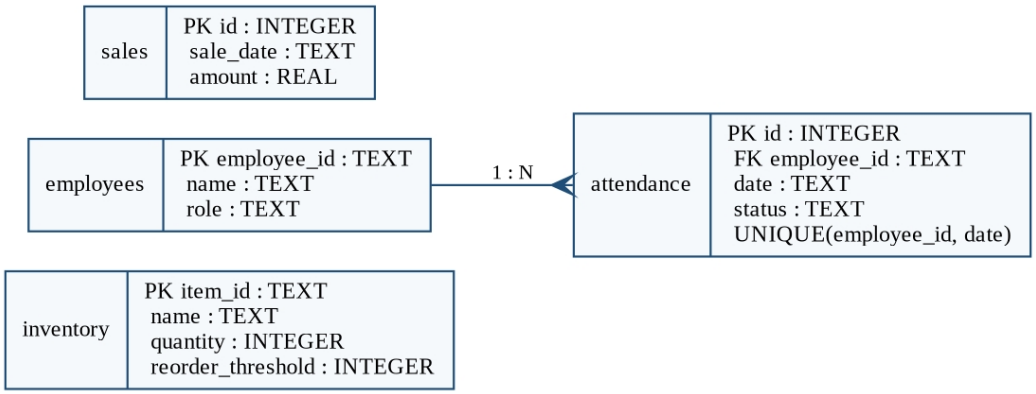


Figure 3.3: Schema of the mock ERP database

Table 3.3: Sample inputs and expected outputs

Input command	Intent	Expected output
Show stock for item P-102	check_stock	Hydraulic Valve (P-102) has 8 units in stock.
Check stock (then "P-101")	check_stock (clarification)	Which item? Please give me the item ID, like P-102. → Steel Bolt 10mm (P-101) has 540 units in stock.
Mark my attendance (then "yes")	mark_attendance (write)	Confirmation prompt, then "Attendance marked as present for E-001 on <date>."
Get total sales from last month	get_sales_report	Total sales last month: 399,500.00, across 3 transactions.
Remove 5 units from P-101	update_stock (write)	Confirmation prompt, then the old and new quantities.
Am I eligible for exams based on attendance?	check_exam_eligibility	Eligibility verdict with attendance % against the 75% threshold (best with the LLM NLU).
Check stock for hydrolick valv	check_stock (fuzzy)	"I think I heard Hydraulic Valve (P-102) — is that right?"

d. Description of Modules

Table 3.4: Modules of the system

Module	File	Description
ERP Core (database)	app/database.py	Creates and seeds the SQLite tables; provides get_stock, get_inventory, get_sales_report, mark_attendance, update_stock, get_attendance_summary, check_exam_eligibility and dashboard queries.
Intent catalogue	app/intents.py	Declares each intent, its required entities and whether it is a write operation.

Module	File	Description
Rule-based NLU	app/nlu.py	Regex-based intent classifier and entity extractor returning an NLUResult (intent, entities, confidence).
LLM-based NLU	app/nlu_llm.py	Sends the sentence to a local Ollama model with a JSON-only prompt; falls back to the rule-based engine if Ollama is unavailable.
Entity Resolver	app/entity_resolver.py	Fuzzy-matches spoken item or employee names to ERP records and returns a confidence score; low-confidence matches are flagged for confirmation.
Dialog Manager	app/dialog_manager.py	Maintains per-session state; handles greetings, missing-entity clarification, misheard-entity confirmation and yes/no confirmation for writes.
ERP Gateway	app/erp_gateway.py	Enforces role permissions (field worker, finance, admin), dispatches intents to ERP functions and formats replies.
Authentication	app/auth.py	Optional PBKDF2-hashed employee sign-in, demo mode and HttpOnly session cookies.
Speech-to-Text	app/stt.py	Pluggable Whisper (API/local) and Vosk engines, plus ffmpeg preprocessing to 16 kHz mono.
Text-to-Speech	app/tts.py	Offline speech synthesis with pyttsx3.
API server	app/main.py	FastAPI routes: /, /health, /auth/*, /erp/options, /erp/inventory, /dashboard/summary, /voice/text, /voice/audio.
Web interface	static/index.html, style.css, app.js	Sign-in screen, chat interface with text and Record button, dashboard, inventory browser with search and low-stock filter, and browser-stored conversation history.
Demo script	demo.py	Runs all use cases from the command line with no setup and then opens an interactive prompt.

e. UML Diagrams

The following UML diagrams describe the system from the perspectives of its users, its structure and its runtime behaviour.

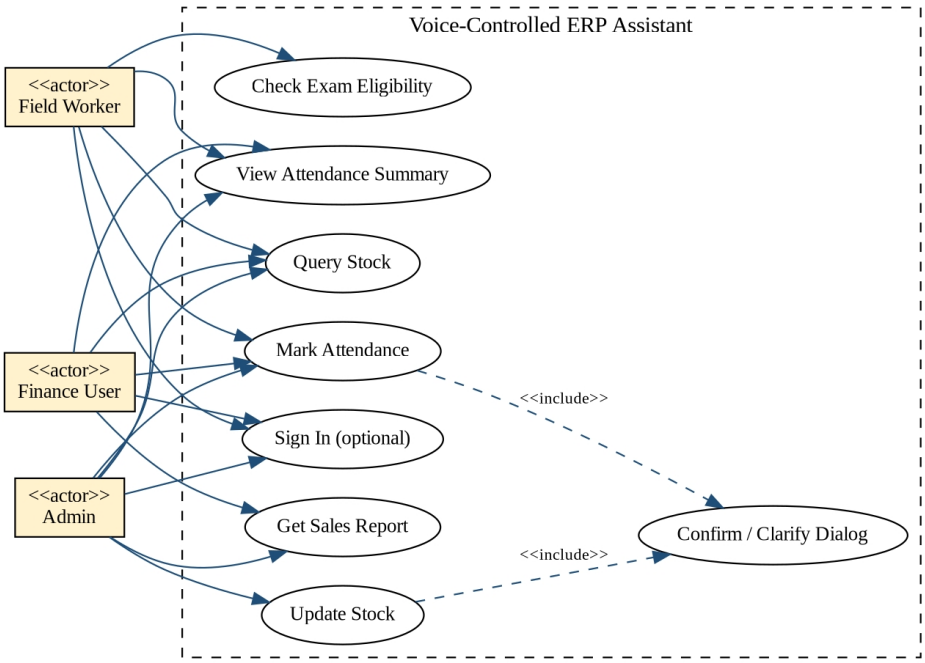


Figure 3.4: Use case diagram

Role permissions in the ERP Gateway determine which use cases each actor can perform: field workers can query stock and manage only their own attendance; finance users can additionally view sales reports; and admins can perform all operations including stock updates. Write operations always include the confirmation dialog.

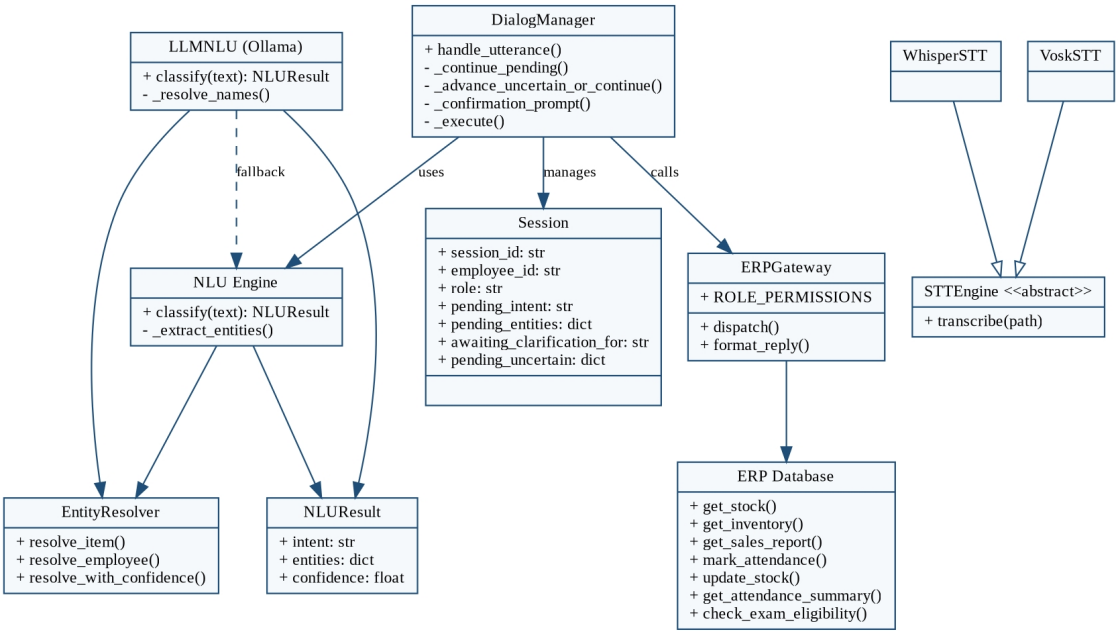


Figure 3.5: Class diagram of the main components

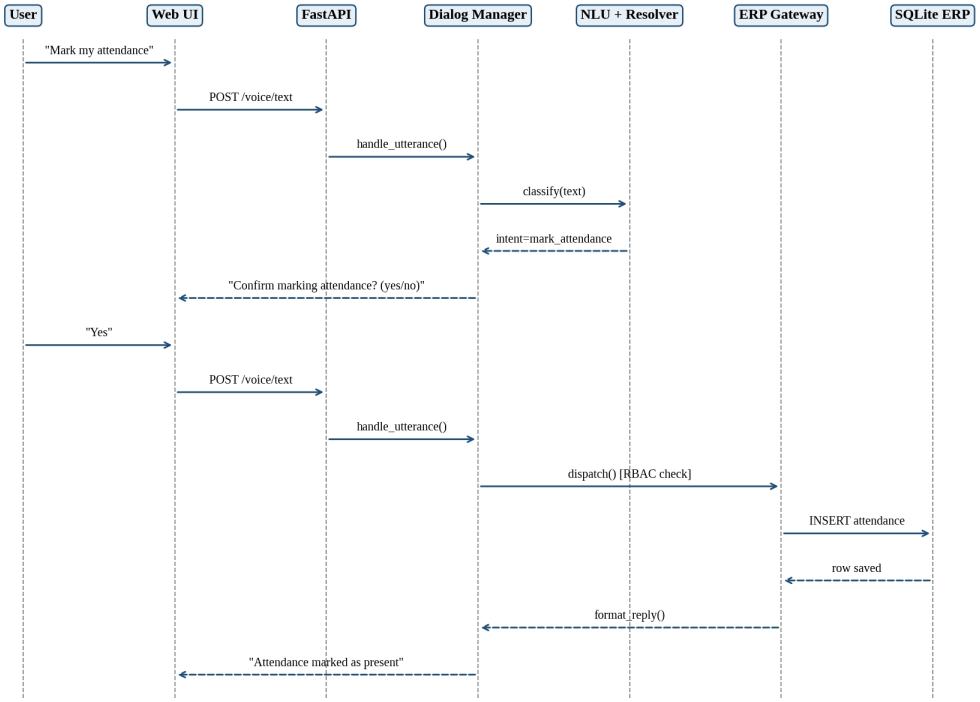


Figure 3.6: Sequence diagram for the write-confirmation flow ("Mark my attendance")

4. Initial Implementation

A working prototype has been implemented and tested end to end for the text pipeline, using the command-line demo (demo.py) and the web interface served by FastAPI. The voice path (Record button, /voice/audio endpoint) is implemented and wired to pluggable STT and TTS engines, and requires the optional dependencies to be installed.

Implemented features

- Mock SQLite ERP with inventory, employees, attendance and sales tables and seed data (three inventory items, three employees with field-worker, finance and admin roles, about 20 days of attendance history, and sample sales transactions).
- Six intents: check_stock, update_stock, mark_attendance, get_attendance_summary, check_exam_eligibility and get_sales_report.
- Rule-based NLU and an optional Ollama LLM-based NLU with automatic fallback.
- Fuzzy entity resolution with confirmation of uncertain matches ("I think I heard X — is that right?").
- Dialog Manager with clarification loops, write confirmation, and small-talk handling (greetings, help, thanks).
- ERP Gateway with role-based permissions and a rule preventing field workers from accessing other employees' attendance.
- Optional PBKDF2-based sign-in with HttpOnly session cookies, and an explicit demo mode.
- Web interface with chat, voice recording, operations dashboard (inventory totals, reorder alerts, monthly sales for authorised roles, attendance summary), an inventory browser, and locally stored conversation history.

Sample execution results

The following transcript was produced by running the demo script against the seeded database:

```
You: Show stock for item P-102
Assistant: Hydraulic Valve (P-102) has 8 units in stock.

You: Mark my attendance
Assistant: Mark attendance as present for employee E-001. Confirm?
(yes/no)
You: yes
Assistant: Attendance marked as present for E-001 on 2026-09-29.

You: Get total sales from last month
Assistant: Total sales last month: 399,500.00, across 3
transactions.

You: Check stock
Assistant: Which item? Please give me the item ID, like P-102.
```

```
You: P-101
Assistant: Steel Bolt 10mm (P-101) has 540 units in stock.
```

Observations

- The four demonstrated use cases (inventory query, attendance with confirmation, sales report and clarification flow) work end to end.
- The rule-based NLU is fast and deterministic but only recognises pre-defined phrasings; the LLM-based NLU is intended to handle natural variation such as "How much attendance do I have?".
- Allowing names instead of codes, together with a confirmation step for shaky matches, reduces the impact of alphanumeric-code recognition errors.

Remaining work

- Evaluate NLU accuracy and speech-recognition word-error rate on a test set of commands, comparing rule-based and LLM-based engines.
- Measure end-to-end response latency for text and voice paths.
- Expand intents and ERP modules; replace the mock database with an Odoo or PostgreSQL-backed ERP through the same function interface.
- Strengthen security (OAuth2/JWT, rate limiting), add multilingual support and proactive alerts, and containerise the application with Docker.

5. References

[1] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey and I. Sutskever, "Robust Speech Recognition via Large-Scale Weak Supervision," arXiv:2212.04356, 2022.

[2] A. Grattafiori et al. (Meta AI), "The Llama 3 Herd of Models," arXiv:2407.21783, 2024.

[3] D. Jurafsky and J. H. Martin, Speech and Language Processing, 3rd ed. (draft), Stanford University, 2025. Chapters on dialogue systems and chatbots.

[4] S. Ramírez, "FastAPI documentation." https://fastapi.tiangolo.com

[5] Alpha Cephei, "Vosk Speech Recognition Toolkit." https://alphacephei.com/vosk/

[6] Ollama, "Ollama documentation and model library." https://ollama.com

[7] SQLite Consortium, "SQLite documentation." https://www.sqlite.org/docs.html

[8] Python Software Foundation, "difflib — Helpers for computing deltas," Python 3 Standard Library documentation. https://docs.python.org/3/library/difflib.html

[9] OWASP Foundation, "Password Storage Cheat Sheet" (PBKDF2 guidance), OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org

[10] pyttsx3, "Offline text-to-speech library for Python." https://pypi.org/project/pyttsx3/